



Lab Manual

Embedded System Design for Machine Learning

Prof. Dr.-Ing. Ulf Schlichtmann
Hon.-Prof. Dr.-Ing. Wolfgang Ecker

Winter semester 2024/25

Teaching Assistants: Philipp van Kempen, M.Sc. &
Samira Ahmadifarsani, M.Sc.

Contents

0. Introduction and Preparation	1
1. Model Design, Training and Quantization	2
2. Model Deployment to an Embedded Platform	3
2.1. Introduction	3
2.2. Background	3
2.2.1. Posterior Handling	4
2.2.2. Performance Evaluation	5
2.2.3. TVM	5
2.2.4. ESP-IDF Toolchain	6
2.2.5. Lab Hardware	7
2.2.6. Lab Software	11
2.3. Assignments	14
2.3.1. Model Optimization and Code-Generation using TVM	14
2.3.2. Microcontroller Deployment using ESP-IDF and MicroTVM	14
2.3.3. Deliverables	16
2.3.4. Testing implementation before final submission	16
2.3.5. Hints	16
2.3.6. Common Problems	17
3. Vectorized Multiply	18
A. Mathematical Background	19

List of Figures

2.1. Posterior Handling	4
2.2. ESP32-C3	8
2.3. INMP441 block diagram	9
2.4. ESP32-C3 Connection Diagram	10
2.5. Micro-KWS Debugger	11
2.6. ESP-IDF Menuconfig Interface	12
2.7. MicroKWS Posterior Unittest	16

Lab 0.

Introduction and Preparation

Lab 1.

Model Design, Training and Quantization

Lab 2.

Model Deployment to an Embedded Platform

2.1. Introduction

In the previous lab, you designed a convolutional neural network (CNN) model architecture using Keras for a simple keyword spotting (KWS) task. After the TensorFlow-based training procedure, the model was quantized and converted to the TFLite format. This format is suitable for embedded devices with limited memory and computing resources. In lab 2, we are going a step further. You will learn how to actually deploy a TFLite model to a real microcontroller device to run real-time inferences with an I2S microphone connected to an ESP32-C3 development board.

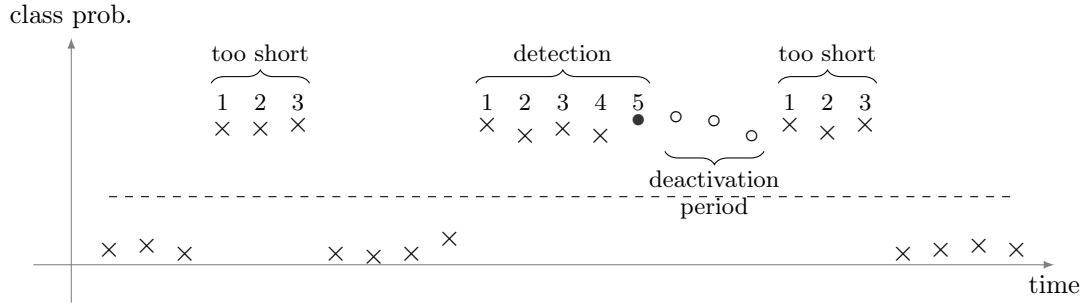
This lab consists of two parts:

1. Generating neural network kernels for the MicroKWS model using the TVM deep learning compiler framework. This involves the generation of C-code for target-optimized neural network inference.
2. Deployment of the MicroKWS code to a RISC-V hardware platform (the ESP32-C3 microcontroller) using a provided target software template.

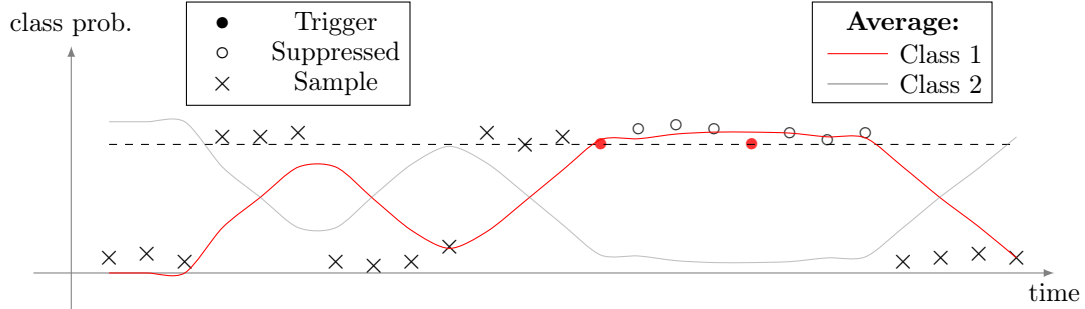
At the end of this manual, you will find a list of assignments you have to complete before the deadline to achieve a good lab grade.

2.2. Background

In lab 1 you were introduced to the basic concepts of keyword spotting and the general keyword spotting pipeline. More specifically, we looked at preprocessing of the audio signal and subsequent frequency domain feature extraction using the Fourier transform. In this lab, we will build on this knowledge and briefly cover the theoretical aspects of posterior handling, i.e. how to process the results coming from the CNN model. The content presented here is closely related to the material presented in lecture 3 *ML Model Design*.



(a) Count based mechanism needing 5 consecutive positive (i.e. larger than the threshold) samples to trigger a detection.



(b) Moving average based mechanism using a window size of 4. If the average value exceeds the threshold, a detection is triggered.

Figure 2.1.: Exemplary class posterior sequence and detection mechanisms. In both cases, we disable the class for 3 samples to prevent repeated triggering.

2.2.1. Posterior Handling

In order to derive a final decision of whether a keyword is present in an incoming audio stream (for example, from a small microphone as presented in Section 2.2.5) one needs to further process the posteriors $\mathbf{a}^L$ yielded by the ML model.

In the previous lab exercises, you trained your model in what is called *non-streaming mode* [1]. You were doing a standard multi-class classification of independent input audio files, which each contained a single keyword. You trained your model on these 1s long audio vectors and observed the (hopefully) increasing accuracy with which your model was able to classify them. Your model operated on well-defined and isolated class realizations (i.e. the audio sample files). However, KWS, in general, is not a static task but a dynamic one. For this reason, the input samples in the training process have been shifted randomly in the time domain and some additive background noise was added. For the deployment, we still need to adjust the code and the non-streaming model such that it can work in what is called *streaming mode*. Dynamic streaming mode systems, in comparison to their non-streaming mode counterparts, do not only need to maintain an acceptable inference frequency, but also have to deal with inter-class transitions,

overlapping speech samples, and partially spoken words. Our system needs to be robust enough to deal with these real-world phenomena. An example algorithm which is going to be implemented in the Lab 2 Assignment is introduced in the following.

Example Algorithm:

The CNN KWS model yields a time series of raw posterior vectors $\{\dots, \mathbf{a}_{t-1}^L, \mathbf{a}_t^L, \mathbf{a}_{t+1}^L, \dots\}$ where t is the discrete time index [1]. The values in this series have strong local correlations and are inherently noisy.

In order to extract a final detection result, we compute a moving average over the raw posterior vectors on a class by class basis. We denote the averaged posterior vector as $\bar{\mathbf{a}}_t^L$, where each entry denotes the averaged classification result for a single word class. This can now directly be used to determine the top-scoring class by simply choosing the maximum component index i of the averaged vector

$$\text{TopClass} = \arg \max_i \bar{\mathbf{a}}_t^L. \quad (2.1)$$

However, since consecutive input feature matrices $\{\dots, \mathbf{X}_{t-1}, \mathbf{X}_t, \mathbf{X}_{t+1}, \dots\}$ extracted from the incoming audio stream might contain multiple fragments of the same keyword realization, false alarms may occur as a result of detecting the same keyword multiple times in the smoothed posterior sequence $\{\dots, \bar{\mathbf{a}}_{t-1}^L, \bar{\mathbf{a}}_t^L, \bar{\mathbf{a}}_{t+1}^L, \dots\}$. This can be circumvented by disabling a just triggered class for a particular short amount of time (see Figure 2.1).

2.2.2. Performance Evaluation

In order to measure the performance of not only the CNN model from the previous lab in non-streaming mode but of the complete system, we need to conduct holistic performance evaluations. They should start at the continuous input audio stream $\mathbf{s}$ and evaluate the final detection result, e.g. ‘Yes’ or ‘No’. This can be done in a variety of ways using many different metrics and scores. For this semester and this particular incarnation of the course, we will not go into any further details here in the lab. Thus, we refer you to the above-mentioned lecture 3 “ML Model Design”, which covers the most relevant metrics. Additionally, section IX. *Evaluation Metrics* of [1] offers a great overview with direct applicability to KWS. For a comprehensive treatment of the subject, we would like to point you to the course *Pattern Recognition* offered by Prof. Rigoll, where such performance metrics are treated in greater detail.

2.2.3. TVM

TensorFlow Lite for Microcontrollers (TFLM) is the software library that revolutionized the the creation of deep learning models for resource constrained embedded systems. TFLM takes advantage of vendor-specific optimized operator libraries to execute the

model on the target device of interest efficiently [2]. With the need to bring deep learning to a wide range of devices and the numerous functions to optimize, the significant engineering effort required to develop these libraries becomes apparent. Therefore, driven by the necessity to bring efficient deep learning accelerations on various platforms, a research team at the University of Washington developed TVM [3], a compiler stack to generate optimized code from deep learning models. TVM is a model deployment framework that offers good performance across a wide range of models on traditional operating systems.

This is achieved by introducing two intermediate representations (IRs) used to describe a model on multiple layers of abstraction. This creates room for optimizations on lower abstraction layers.

- **Relay** is the high-level functional IR used to represent full models. Here, graph-level optimizations can be included, allowing to offload complete functions (sub-graphs) to an accelerator device. Example operators in RelayIR are `nn.dense` or `qnn.conv2d`
- **TIR** (Tensor Intermediate Representation) contains the definition of the low-level program representations. So-called *Compiler Passes* can be used to transform these programs composed of basic operations such as `tir.round` or `tir.log`.

A given (pre-quantized) model in the TFLite format can be translated to Relay IR by one of the numerous frontends provided by TVM. Then various compiler phases are applied in a process called *Lowering* to create a TIR description of the model. To finally deploy these optimized models to run on a specified target, TVM provides code-generation backends, i.e. for C, LLVM or even accelerator-specific configuration code.

For deployment of models to microcontroller-type devices, microTVM can be used, as it provides interfaces for building, flashing and running models using a microTVM project API server with minimal efforts required. As the microTVM subproject is still in development, we will not use these features in this lab and instead deploy a custom MicroKWS target software wrapping around the C-kernels generated by TVM using the ESP-IDF toolchain directly. Bare-metal implementations are possible, however for interfacing with sensors and other peripherals during inference, it is often a good choice to use a real-time operating system (RTOS). With the help of the *Ahead-of-Time* (AoT) compiler, microTVM does not even require a runtime environment to be present on the target device, further reducing compute and memory overhead. The generated kernels from (micro)TVM, i.e. the C code, can then simply be included in the ESP-IDF project.

2.2.4. ESP-IDF Toolchain

In order to compile, upload, and debug the ESP32-C3 chip, we are using the Espressif IoT Development Framework (ESP-IDF). The ESP-IDF is a free and open-source development framework provided and maintained by the chip manufacturer Espressif, as

well as an open-source community. It builds on-top of the FreeRTOS real-time operating system and provides drivers for the peripherals supported by these chips. The codebase is hosted on GitHub [4] and offers a decent documentation [5].

When looking through the documentation, ensure that you select the correct chip (i.e. ESP32-C3), as well as the correct ESP-IDF version (i.e. `release/v5.2`) from the dropdown menus in the top-left corner.

Installation

The ESP-IDF offers support for all three major platforms (Windows, Linux, and macOS). Please follow the steps listed under <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/get-started/index.html#manual-installation> in order to install the ESP-IDF. **Please install release/v5.2.** We highly recommend that you run the ESP-IDF on Linux. You will only need to follow steps 1. through 4. of the Linux installation guide. After this, you should have the tools installed and be ready for your first tests. Should you run into any issues, please let us know, and we will assist you as good as possible.

2.2.5. Lab Hardware

For this lab, you will receive a small box (aka. lab kit) with a number on it and a couple of electronic parts inside. Depending on the number of students participating, we might not be able to offer a single lab kit to every student.

The lab kit contains the following hardware parts:

- An ESP32-C3 development board
- An INMP441 digital I2S microphone
- A breadboard
- 10 breadboard wires
- A micro USB cable

As with any physical objects, the contents of the box (and the box itself, for that matter) can be damaged if not handled correctly. In order to be able to offer the lab more than once, we would kindly ask you to treat the hardware you received with respect. Please be careful and double-check your connections. If you, for example, short power to ground, you will most likely burn out the onboard voltage regulator. That being said, we want you to know that you can contact us anytime should you feel that the hardware is not working as it should. Please don't hesitate to get in touch with us if you suspect that you might have broken something. We will figure out where the issue might be and provide spare parts if needed. Being honest with us won't affect your pass/fail grade in the lab in any way or form. We have plenty of cleanup work at the chair kitchen that you can do in order to compensate for the broken parts. ☺

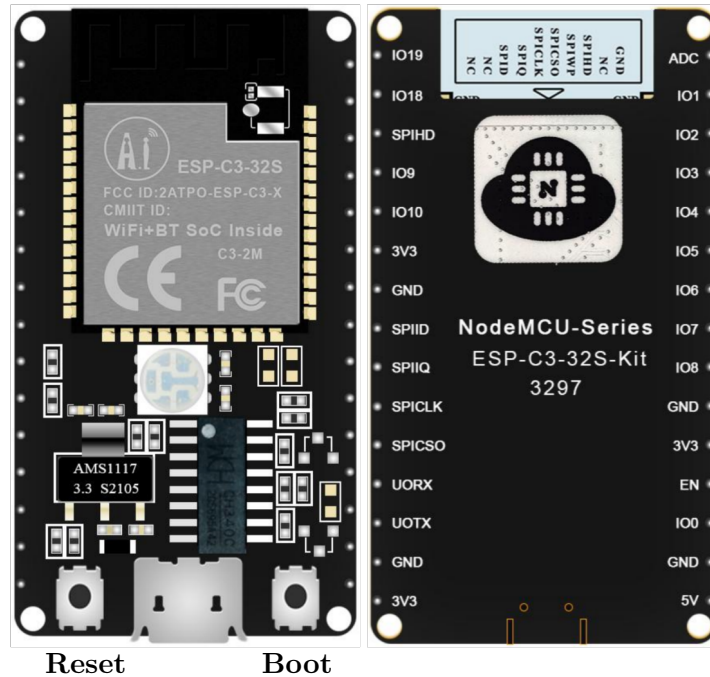


Figure 2.2.: ESP32-C3 development board from top-down (left) and bottom-up (right).

ESP32-C3 Microcontroller

The ESP32-C3 is a single-core, 32-bit, RISC-V-based microcontroller with 400KB of SRAM. It is fabbed in a 40nm CMOS technology. It is capable of running at 160MHz and has integrated 2.4 GHz Wi-Fi and Bluetooth 5 (LE). Additionally, it features 22 programmable GPIOs with support for ADC, SPI, UART, I2C, I2S, CAN, and PWM.

The ESP32-C3 has a RV32IMC core (**I**nteger, **M**ultiply, **C**ompressed) and features:

- an in-order four-stage pipeline that supports a clock frequency of up to 160 MHz,
- a 32-bit multiplier and a 32-bit divider,
- up to 32 vectored interrupts at seven priority levels,
- up to 8 hardware breakpoints/watchpoints,
- and up to 16 PMP regions.

With these specs, the ESP32-C3 is a fairly capable microcontroller. While it is the first RISC-V based chip offered by Espressif, they have just recently announced that they will be moving their entire microcontroller lineup from the proprietary Tensilica Xtensa ISA exclusively to the RISC-V open-source ISA.

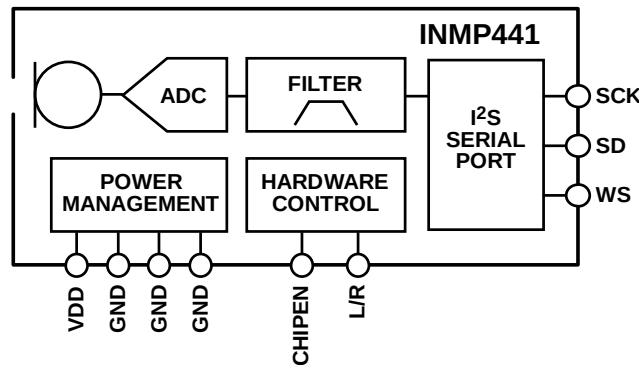


Figure 2.3.: INMP441 microphone functional block diagram [6].

For this lab we will be using an ESP32-C3 development board that features the ESP32-C3 chip (RV32-IMC), as well as 2MB of external Flash storage, an oscillator, and some minor passive components (see Figure 2.2). Additionally, the module has a small PCB antenna, a metal RF shield, and a USB adapter. However, for this lab we won't be using any of the peripherals, except the LEDs and the I2S interface.

INMP441 Microphone

The INMP441 is a high-performance and low power, digital MEMS microphone. It is a SMD mounted part with the audio port tunneling through the PCB. The microphone chip features a MEMS audio sensor, signal conditioning circuits, an analog to digital converter, an anti-aliasing filter and power management components (see Figure 2.3). The microphone can be connected via an industry standard 24-bit I2S interface. The I2S interface allows the INMP441 to directly connect to microcontrollers without the need for an external analog-to-digital converter. While the INMP441 has many great features, we will only be using it in a fairly straightforward configuration: single-channel (mono) with a 16kHz sample rate and 16bit resolution.

The driver to interface with the INMP441 via I2S is already implemented in the target software. Feel free to have a look at it and try to get a basic understanding of how it works. However, you generally don't need to worry too much about it and we won't expect you to know the details.

Connections and Hardware Setup

During your studies, you should ideally have come in contact with breadboards and jumper wires before. Thus, we will not go into any details on how to use them. All you need to do is to place both the ESP32-C3 breakout board and the INMP441 microphone onto to the breadboard and connect the components according to the schematic shown

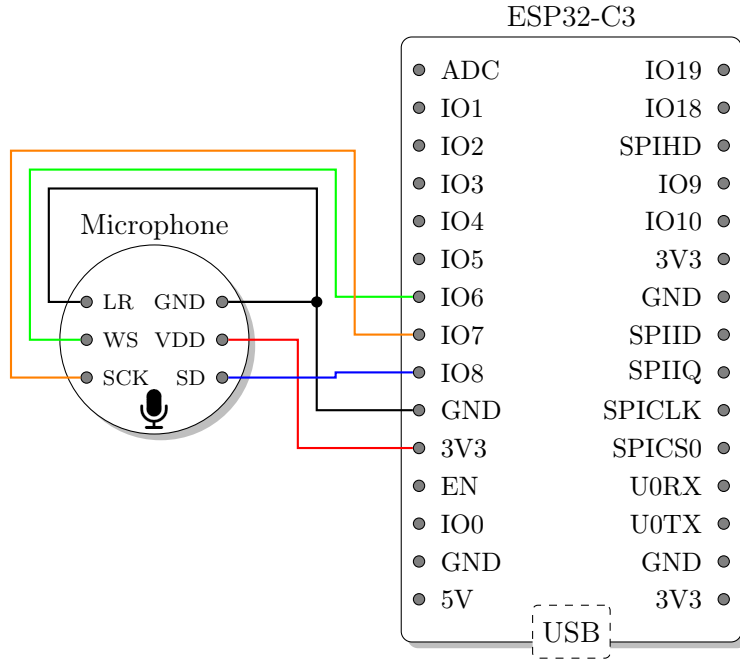


Figure 2.4.: Top-down view of the I2S connection diagram with ESP32-C3 development board and the INMP441 microphone.

in Figure 2.4. As mentioned above, please double-check your connections. Pay special attention to the GND and VCC (3.3V) connections, as connecting them incorrectly can lead to irreversible damage to the chip or the microphone.

First Test

Use the provided micro USB cable to connect the ESP32-C3 to your computer. On Linux and macOS machines, you should see a new device appear. Use `ls /dev/tty*` for Linux, and `ls /dev/cu.*` on macOS. On Windows you should see a new COM-Port appear in the device manager. You might need to install some drivers on Windows. More info on serial connections with the ESP32-C3 on all three platforms can be found here <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/get-started/establish-serial-connection.html>.

Once you have set up your connection, you are ready to compile and upload your first example program to the ESP32-C3. Please refer to Section 2.3.2 on how to upload the target software to the chip.

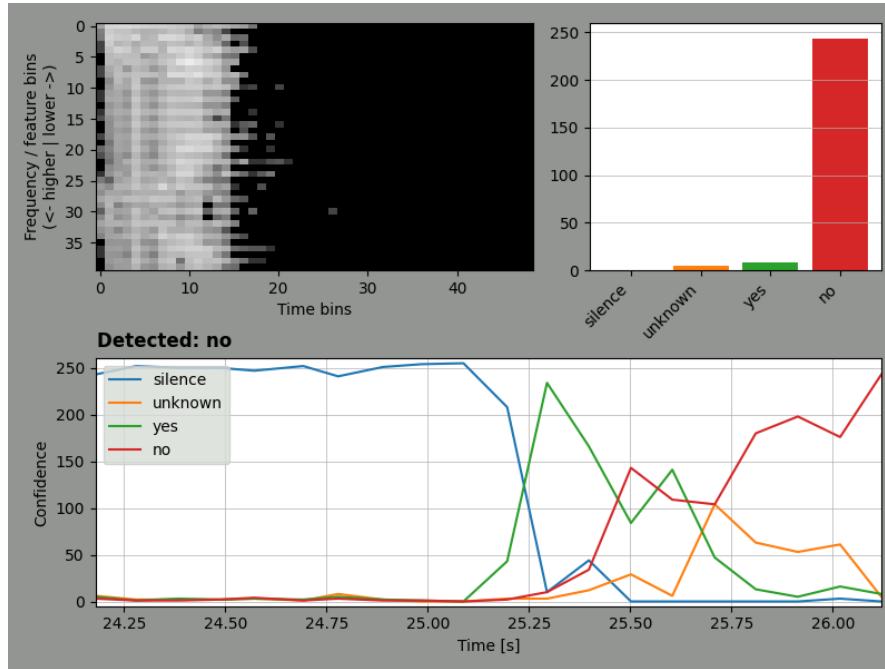


Figure 2.5.: The micro-KWS Python debugger GUI.

2.2.6. Lab Software

MicroKWS Overview

MicroKWS Debugger

The ESP32-C3 target software by default prints a number of useful, but limited, debugging information over the serial port in human-readable ASCII format. It includes a raw view of the neural network posteriors, as well as the time delta between inferences. However, we also provide a *debugging mode*. In this mode, the ASCII prints are disabled and instead binary debugging data is transmitted over the serial port to the host PC. In conjunction with the small Python program we wrote, you can visualize this debugging data in a GUI (see Figure 2.5).

In the top-left corner of the debugger GUI, you have a live view of the feature matrix that the convolutional neural network (CNN) uses for keyword detecting. This is the output of the feature-extraction frontend. In the top-right corner, you can see the current posterior values output by the CNN as a bar plot. This graph visualizes the instantaneous result, while the large history plot in the lower half of the GUI displays the results over time. Above the top-left corner of the history plot, you can see the current top category determined by the posterior post-processing (aka. *the backend*). This is the final output of the KWS detection system.

```

(Top) → MicroKWS Options
Espressif IoT Development Framework Configuration
MicroKWS Hyperparameters --->
(100) Maximum number of inferences per second
MicroKWS Posterior Handler Parameters ---->
(10) Number of classes
MicroKWS Class Labels ---->
MicroKWS Debug Settings ---->

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(a) MicroKWS Config Menu

```

(Top) → MicroKWS Options → MicroKWS Class Labels
Espressif IoT Development Framework Configuration
(silence) MLF Class 0 Label
(unknown) MLF Class 1 Label
(yes) MLF Class 2 Label
(no) MLF Class 3 Label
(???) MLF Class 4 Label (NEW)
(???) MLF Class 5 Label (NEW)
(???) MLF Class 6 Label (NEW)
(???) MLF Class 7 Label (NEW)
(???) MLF Class 8 Label (NEW)
(???) MLF Class 9 Label (NEW)

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(b) Submenu for configuring the label names

Figure 2.6.: ESP-IDF Menuconfig Interface (`idf.py menuconfig`)

In order for the binary debugging data needed by the KWS debugger to be transmitted over serial, you will need to modify a couple of settings in the micro KWS target software. Please have a look at the `README.md` in the `debug/` directory. It has plenty of information on how to get started with the different debugging modes and the debugger desktop GUI.

MicroKWS Configuration

There are two approaches to configure the ESP-IDF project:

- Manually editing the `sdkconfig` file in the project directory.
- Using the command `idf.py menuconfig`

The Menuconfig tool is a command line-based interface and should be more user-friendly than the file-based configuration as it arranges all settings in nested menus. We added a custom submenu which can be found in the top-level page of the Menuconfig interface. Here you can adapt various settings as shown in Figures 2.6a and 2.6b. After changing the configuration in either way, a recompilation of the whole ESP-IDF project is required.

2.3. Assignments

You will find the all relevant code and documentation for this assignment in the `micro-kws` repository hosted on the TUM GitLab server: <https://gitlab.lrz.de/de-tum-ei-eda-esl/micro-kws>. Please clone this repository and ensure you are on branch `lab2`.

2.3.1. Model Optimization and Code-Generation using TVM

The points listed below provide a high-level view of the steps you will need to follow in order to complete the first part of the lab 2 assignment. Please also refer to the `tvm/README.md` file for more information.

Tasks

- Prerequisites:
 1. Install the TVM ML compiler suite (follow the instructions listed in the `2_deploy/install_tvm.md` and `2_deploy/whl/README.md`).
 2. Verify your TVM installation.
- Generate kernels for the provided MicroKWS model (using 8 keywords) with the TVM ML compiler flow:
 1. Read and follow the steps 1. and 2. listed in the `tvm/README.md` file. They point you to two other markdown files:
 - TVMC Command-Line (`2_deploy/tutorial_tvmc.ipynb`)
 - MLF Overview (`2_deploy/mlf_overview.md`)
 2. At the end of this section you should at least have the following files:
 - (A) `2_deploy/gen/mlf.tar`
 - (B) `2_deploy/gen/mlf_tuned.tar`

2.3.2. Microcontroller Deployment using ESP-IDF and MicroTVM

In the second half of this lab you will deploy your kernels onto the microcontroller board with the help of the MicroTVM and ESP-IDF.

Using a chair computer

If you are working on a chair computer, you don't have to install the ESP-IDF toolchain by yourself. Instead, consider the following:

- Use your personal working directory in `/usr/local/labs/ESD4ML/current/` instead of your users home directory to avoid any issues with the disk quota.
- Source the `setup.sh` script to set up your shell environment in each terminal window:

```
source /usr/local/labs/ESD4ML/current/common/scripts/setup.sh
```

Important note The installation of the ESP-IDF toolchain is shared by all the students and therefore read-only! Please do not try to update or delete these files.

Using your own computer

Follow the instructions in the `micro-kws` repository on how to install the ESP-IDF toolchain on your OS (see `3_run/README.md`).

Note on virtual environments Sourcing the ESP-IDF toolchain will result in activating a virtual environment. Thus, stacking the active ESP-IDF on top of a virtual environment or vice-versa should be avoided. If you want to use a virtual environment while working with the `idf.py` tool, please source the virtual environment in a new terminal session instead.

Tasks

- Prerequisites:
 1. Ensure that the ESP-IDF toolchain is available (correctly installed and sourced).
 2. Verify that hardware is working properly.
- Deployment:
 1. Follow MicroTVM (ESP-IDF) tutorial in the `3_run` directory.
 2. Generate two project directories using the MicroTVM command-line interface (`tvmc micro create ...`)
 - a) Using the untuned (`mlf.tar`) TVM artifacts: `3_run/prj/`
 - b) Using the tuned (`mlf_tuned.tar`) TVM artifacts: `3_run/prj_tuned`
 3. Perform the following tasks for both of the MicroTVM projects:
 - a) Update the project settings to use the new model architecture (e.g. number of keywords, label names etc.).
 - b) Compile, flash and run the demo program. How does it perform?
 - c) Determine the static ROM and RAM usage of this program using the `idf.py size` command. Can you explain why the software size exceeds the estimated metrics ¹ of the model?

For the following, only consider the tuned variant.

 4. Solve programming challenges:
 - Implement the moving-average based post-processing introduced in Section 2.2.1 for the MicroKWS software by editing the files
 - * `3_run/student/posterior/posterior.cc`

¹Using the memory metrics from lab 1, we estimate the model to be: $ROM \approx 27\text{ kB}$, $RAM = 39\text{ kB}$

```
* 3_run/student/posterior/include/posterior.h
```

- Use the `idf.py menuconfig` command to modify the algorithms parameters (history lengths, detection threshold, deactivation period) which should **not** be hardcoded in the C++ file.

2.3.3. Deliverables

- **Moodle Upload:**

- Single ZIP archive: `submission_2.zip` (generated using `3_run/submit.py`)

2.3.4. Testing implementation before final submission

We provide a small unit test for the to be implemented posterior module which can be used to check if your code matches the requirements. The test does not cover every edge case, thus passing the test does not guarantee a correct solution. Details on how to run the test on the MCU (or a linux host) are provided in `3_run/student/posterior/test/README.md`.

```
(128) main_task: Started on CPU
(128) main_task: Calling app_main()

### Running all tests ###
Running History length: 35, Threshold: 100, Suppression: 0 ms...
/nas/ei/share/TUEIEDA/LabES04ML/ws2324/ga87py/micro-kws-private/3_run/student/posterior/test/test_posterior.cpp:103:History length: 35, Threshold: 100, Suppression: 0 ms:FAIL: Expected 0 Was 1
Running History length: 35, Threshold: 100, Suppression: 100 ms...
/nas/ei/share/TUEIEDA/LabES04ML/ws2324/ga87py/micro-kws-private/3_run/student/posterior/test/test_posterior.cpp:238:History length: 35, Threshold: 100, Suppression: 100 ms:FAIL: Expected 0 Was 1
-----
2 Tests 2 Failures 0 Ignored
FAIL
(1540) main_task: Returned from app_main()
```

Figure 2.7.: MicroKWS Posterior Unittest Output.

Figure 2.7 shows how the unit testing outputs may look like. Here two out of two tests are failing. In this case you would need to find the bugs in your implementation to pass the tests.

2.3.5. Hints

- Only modify the code sections which are highlighted as follows:
`/* ENTER STUDENT CODE BELOW */`
- Inline comments and docstrings are provided in the respective C++ files to help you with your implementation.
- To check the functionality of your code before the final submission, some basic unit tests are provided. (See `3_run/prj_tuned/tests/README.md` for details)
- The goal of this lab is not to get a perfect detection accuracy but rather to learn how the real-time inference of a simple keyword-spotting task on a *real hardware device* introduces new challenges we have to consider.
- You are allowed to change the setting of the posterior handler (see `idf.py menuconfig`) to adapt the detection performance to the inference latency of the used model.

2.3.6. Common Problems

- Windows/MacOS related Issues

Solution: These systems are not completely supported and untested, please use a student PC or Ubuntu 20.04 on your own computer (i.e. in a VM)

- TVM prints warnings to the command line:

```
Warning: Desired layout(s) not specified for op: nn.max_pool2d
```

Solution: These warnings can be safely ignored.

- ESP-IDF not working inside an IDE (i.e. VSCode) **Solution:** Add the definitions of IDF_PATH and IDF_TOOLS_PATH as well as the source \$IDF_PATH/export.sh to your users bash configuration file (i.e. \$HOME/.bashrc).

- ESP32-C3 reports wrong boot mode during flashing

Solution: Press and hold the button right next to the USB port of the ESP32-C3 board until the flashing completed.

- Ubuntu: Accessing the serial port of the ESP32-C3 board not allowed as non-root user.

Solutions:

1. Either configure udev to allow non-root user to access the serial port. Example rule (/etc/udev/rules.d/50-esp32c3-devkit.rules):

```
# QinHeng Electronics HL-340 USB-Serial adapter (ESP32-C3 DevKit)
SUBSYSTEM=="tty" ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", MODE="666"
```

2. Or just modify the permission of the specific device:

```
sudo chmod -R 777 /dev/ttyUSB0
```

- The printed class names are being shown as ??? (see screenshot).

```
Detected a new keyword silence
SILENCE: 255 unknown: 0 yes: 0 no: 0 ????: 0 ????: 0 ????: 0 ????: 0 ????: 0 ????: 0 Δ514ms
Detected a new keyword ???
silence: 1 unknown: 9 yes: 1 no: 1 ????: 70 ????: 0 ????: 0 ????: 0 ????: 7 ????: 167 Δ514ms
Detected a new keyword ???
silence: 0 unknown: 2 yes: 0 no: 0 ????: 28 ????: 0 ????: 0 ????: 0 ????: 21 ????: 206 Δ512ms
Detected a new keyword silence
SILENCE: 255 unknown: 0 yes: 0 no: 0 ????: 0 ????: 0 ????: 0 ????: 0 ????: 0 ????: 0 Δ513ms
Detected a new keyword silence
SILENCE: 255 unknown: 0 yes: 0 no: 0 ????: 0 ????: 0 ????: 0 ????: 0 ????: 0 ????: 0 Δ513ms
```

Solution: Use the idf.py menuconfig interface to configure the correct class names (see Section 2.2.6). If this step is skipped, you will lose points in the assignment! The monitor is expected to look like this:

```
silence: 69 unknown: 5 yes: 2 no: 3 up: 69 down: 1 left: 7 right: 2 on: 7 OFF: 92 Δ514ms
Detected a new keyword up
silence: 0 unknown: 9 yes: 0 no: 2 UP: 202 down: 3 left: 0 right: 0 on: 5 off: 36 Δ514ms
Detected a new keyword silence
SILENCE: 231 unknown: 2 yes: 1 no: 1 up: 10 down: 1 left: 2 right: 1 on: 1 off: 7 Δ512ms
Detected a new keyword silence
SILENCE: 255 unknown: 0 yes: 0 no: 0 up: 0 down: 0 left: 0 right: 0 on: 0 off: 0 Δ514ms
Detected a new keyword down
silence: 0 unknown: 6 yes: 0 no: 1 up: 0 DOWN: 245 left: 0 right: 0 on: 4 off: 0 Δ514ms
```

Lab 3.

Vectorized Multiply

Appendix A.

Mathematical Background

Bibliography

- [1] Ivan Lopez-Espejo et al. “Deep Spoken Keyword Spotting: An Overview”. In: *IEEE Access* (2021).
- [2] Gian Marco Iodice. *TinyML Cookbook: Combine artificial intelligence and ultra-low-power embedded devices to make the world smarter*. Packt Publishing, 2022.
- [3] Tianqi Chen et al. “{TVM}: An Automated {End-to-End} Optimizing Compiler for Deep Learning”. In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 2018, pp. 578–594.
- [4] Espressif and Open-Source Collaborators. *Espressif IoT Development Framework*. 2022. URL: <https://github.com/espressif/esp-idf>.
- [5] Espressif and Open-Source Collaborators. *ESP-IDF Programming Guide*. 2022. URL: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/index.html>.
- [6] *Omnidirectional Microphone with Bottom Port and I2 S Digital Output*. INMP441. Rev. 1.1. InvenSense Inc. May 2014.